
Learning To Sample From Diffusion Models Via Inverse Reinforcement Learning

Constant Bourdrez¹ Alexandre Vérine¹ Olivier Cappé¹

Abstract

Diffusion models generate samples through an iterative denoising process, guided by a neural network. While training the denoiser on real-world data is computationally demanding, the sampling procedure itself is more flexible. This adaptability serves as a key lever in practice, enabling improvements in both the quality of generated samples and the efficiency of the sampling process. In this work, we introduce an inverse reinforcement learning framework for learning sampling strategies without retraining the denoiser. We formulate the diffusion sampling procedure as a discrete-time finite-horizon Markov Decision Process, where actions correspond to optional modifications of the sampling dynamics. To optimize action scheduling, we avoid defining an explicit reward function. Instead, we directly match the target behavior expected from the sampler using policy gradient techniques. We provide experimental evidence that this approach can improve the quality of samples generated by pretrained diffusion models and automatically tune sampling hyperparameters.

1. Introduction

Diffusion models and related generative processes based on differential equations have become a standard tool for modeling high-dimensional data, with applications ranging from image synthesis to scientific and biological modeling. Sample generation proceeds by simulating a reverse-time stochastic or deterministic process that gradually transforms a simple Gaussian distribution into a complex data distribution. This evolution is governed by a vector field implemented by a large neural network, commonly referred to as the denoiser.

While the training of diffusion models has been extensively

studied, the sampling procedure itself remains a critical bottleneck. A growing body of work proposes heuristic modifications of the sampling dynamics, including restart sampling (Xu et al., 2023), guidance mechanisms (Ho & Salimans, 2022; Dhariwal & Nichol, 2021), noise schedule tuning (Lee et al., 2023), and stochasticity injection (Karras et al., 2022). Although often effective, these approaches are typically designed in an ad hoc manner and rely on manually tuned hyperparameters, which makes them difficult to adapt systematically to new objectives, datasets, or computational constraints. In this work, we propose to learn such parameters automatically by framing diffusion sampling as a sequential decision-making problem.

Reinforcement learning has recently been applied to generative modeling, most prominently in the context of large language models through reinforcement learning from human feedback (Wang et al., 2023; Rafailov et al., 2023; Azar et al., 2023; Sun & van der Schaar, 2025). Several works extend this paradigm to diffusion models by defining explicit reward functions and fine-tuning either the denoiser or the sampling process (Black et al., 2023; Fan et al., 2023; Guo et al., 2024; Uehara et al., 2024; Barceló et al., 2024). In parallel, diffusion models have also been studied through the lens of stochastic control. Recent work on adjoint matching and related control-based formulations proposes to fine-tune diffusion or flow models by matching backward dynamics or solving variational control problems in continuous time (Azangulov et al., 2025; Domingo-Enrich et al., 2025; Santi et al., 2025). These approaches are naturally formulated in continuous time and rely on differentiating through the generative dynamics. A related inverse reinforcement learning approach for diffusion models is proposed by Yoon et al. (2024), who learn an energy-based reward to guide sampling.

In this work, we take a different perspective and cast diffusion sampling as an imitation learning problem. Given access to an expert diffusion sampler *e.g.*, samples from the forward diffusion process at different noise levels, we learn a policy that adapts the sampling process of a pre-trained diffusion model to better match expert behavior. While our approach builds on state-marginal inverse reinforcement learning, adapting these ideas to diffusion sampling requires addressing structural constraints that are absent in generic MDPs, including absorbing terminal dynamics, and expert

¹DI ENS, École normale supérieure, Université PSL, CNRS, 75005 Paris, France. Correspondence to: Constant Bourdrez <constant.bourdrez@ens.psl.eu>, Alexandre Vérine <alexandre.verine@ens.psl.eu>.

supervision limited to marginal distributions at selected noise levels. In contrast to prior IRL and control-based approaches for diffusion models, our method (i) operates purely at sampling time, (ii) does not learn rewards or energies, and (iii) exploits the structure of noise levels to define expert occupancy without requiring expert trajectories.

2. Sample Generation with Diffusion Models

Diffusion models generate data in $\mathbb{R}^d$ by transporting samples from a Gaussian distribution to a target data distribution P_0 , via a reverse-time stochastic or deterministic process (Song & Ermon, 2020; Song et al., 2022). The reverse dynamics rely on a neural network F (the *denoiser*) trained across noise levels to estimate the score of the noised data distribution. In practice, F is trained either to predict the added noise (Ho et al., 2020) or the denoised sample (Dhariwal & Nichol, 2021), and may be conditioned on auxiliary information c such as class labels or text prompts.

While diffusion models are commonly described in continuous time, practical implementations rely on a finite discretization. We can see the generation process as a sequence of states $s_t = (x_t, \sigma_t)$, where x_t is the generated sample at time t and σ_t is the corresponding noise level defined by the noise schedule $\{\Sigma_N, \dots, \Sigma_0\}$. Typically, $\Sigma_0 = 0 < \Sigma_1 < \dots < \Sigma_N$ and Σ_N is large enough for P_N to approximate pure Gaussian noise. The sequence starts at $t = 0$ from $s_0 = (x_0, \Sigma_N)$ with $x_0 \sim \mathcal{N}(0, \Sigma_N^2 I_d)$ and ends at $t = T$ with $s_T = (x_T, \Sigma_0)$, where x_T is expected to follow the target distribution P_0 . Each step of the reverse process maps (x_t, σ_t) to (x_{t+1}, σ_{t+1}) according to

$$(x_{t+1}, \sigma_{t+1}) = H(x_t, \sigma_t, \dots),$$

where H denotes a generic update operator that uses the denoiser F , evaluated at the current state (x_t, σ_t) and possibly additional inputs such as conditioning information c . We treat $\sigma_t = \Sigma_0$ as an absorbing condition, i.e., once reached, the process remains at this noise level for all subsequent steps. The operator H can represent a variety of samplers: it is stochastic for DDPM (Ho et al., 2020) and Score-SDE (Song et al., 2021), and deterministic for probability-flow ODE (PF-ODE) based samplers, e.g., first-order Euler’s method (Song et al., 2022), second-order Heun’s method (Jolicœur-Martineau et al., 2021; Karras et al., 2022), or higher-order solvers such as DPM-Solver (Lu et al., 2022). Notably, H depends on hyperparameters that influence the sampling dynamics. In this work, we focus on three popular techniques to modify the sampling process: stochasticity injection, classifier-free guidance, and restart sampling.

Stochasticity Injection: The EDM approach of Karras et al. (2022), which serves as a strong baseline for image generation, employs a hyperparameter γ_{EDM} to introduce controlled stochasticity. Using H_{Heun} to denote the deter-

ministic Heun update operator, the modified update rule relies on an amplified noise level $\hat{\sigma}_t := \sigma_t (1 + \gamma_{\text{EDM}})$ and a noise perturbation $z \sim \mathcal{N}(0, I)$:

$$H_{\text{EDM}}(x_t, \sigma_t) = H_{\text{Heun}}(x_t + \sqrt{\hat{\sigma}_t^2 - \sigma_t^2} z, \hat{\sigma}_t).$$

For some datasets such as CIFAR-10 (Alex Krizhevsky, 2009) or FFHQ (Karras et al., 2019), $\gamma_{\text{EDM}} = 0$ yields optimal results, while for more complex datasets such as ImageNet (Deng et al., 2009), non-zero values (applied at intermediate noise levels) can improve sample quality.

Classifier-Free Guidance: Classifier-free guidance (Ho & Salimans, 2022) is one of the most popular techniques to enhance sample quality in conditional diffusion models. For any H , where the model F has been trained to be conditioned on c or unconditioned ($c = \emptyset$), the update rule is kept identical to H but the denoiser is modified as

$$F_{\text{CFG}}(x_t, \sigma_t, c) = (1 + \omega)F(x_t, \sigma_t, c) - \omega F(x_t, \sigma_t, \emptyset).$$

Depending on the guidance scale ω , this technique can significantly improve sample fidelity, at the cost of diversity (Ho & Salimans, 2022; Saharia et al., 2022).

Restart Sampling: Another approach consists in restarting the sampling process to escape poor local generations (Xu et al., 2023). Let $i < j$ be two integers such that $\Sigma_j > \Sigma_i$. When the sample reaches Σ_i , it is sent back K times to Σ_j by adding a random Gaussian perturbation $z \sim \mathcal{N}(0, I)$. Let k_t denote the number of restarts performed up to time t ; the update rule usually relies on any PF-ODE-based operator H_{ODE} and becomes

$$H(x_t, \sigma_t) = \begin{cases} H_{\text{ODE}}(x_t + \sqrt{\Sigma_j^2 - \Sigma_i^2} z, \Sigma_j), \\ \quad \text{if } k_t < K \text{ and } \sigma_t = \Sigma_i, \\ H_{\text{ODE}}(x_t, \sigma_t) \quad \text{otherwise.} \end{cases}$$

This technique can be generalized in several ways. The variant we consider in this paper allows renoising from any Σ_i to one of the M previous higher noise levels $\Sigma_{i+1}, \dots, \Sigma_{i+M}$ upon reaching a restart level Σ_i . In the experiments, we will fix $M = 4$ and refer to this variant as *Renoise*. Additionally, a fixed budget of function evaluations is used instead of a fixed number of restarts.

3. State Distribution Matching with IRL

In this section, we review reinforcement learning (RL) and inverse reinforcement learning (IRL) tools that motivate our formulation in Section 4. Our goal is to cast the diffusion sampling algorithm from Section 2 as a decision-making problem, in order to select sampler hyperparameters (e.g., guidance scales, injected noise, or renoise rules). Crucially,

we do *not* assume access to an expert policy or expert actions. Instead, we only observe samples from the data distribution and their noisy counterparts across noise levels, which naturally aligns with IRL settings based on matching state distributions.

MDP formulation We formalize diffusion sampling as a finite-horizon Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \rho, T)$. We consider discrete-time policies with a fixed horizon T , initial states sampled from distribution ρ , and transition dynamics defined by the kernel $\mathcal{P}$. The state space $\mathcal{S}$ is continuous, and the action space $\mathcal{A}$ is discrete. Let $\pi_\theta(a_t | s_t)$ be a parametrized policy over actions conditioned on states, and let $\mathbf{s}_{1:T} = (s_1, \dots, s_T)$ denote a trajectory generated by this policy. In Section 4, we will specify how each component of the MDP is instantiated for diffusion samplers.

State occupancy measures In our setting, the expert policy is unknown and we do not observe expert actions or full trajectories; the only supervision comes from expert states (data samples and their noised versions). This state-only regime motivates comparing policies through their *state occupancy measures*, i.e., the marginal distribution over states visited along trajectories.

Definition 3.1 (Occupancy Measure). The policy π_θ induces a state occupancy measure μ_θ defined as

$$\forall \Phi \in \mathcal{B}(\mathcal{S}), \quad \int \Phi(s) \mu_\theta(s) = \mathbb{E}_{\mathbf{s}_{1:T} \sim p_\theta} \left[\frac{1}{T} \sum_{t=1}^T \Phi(s_t) \right],$$

where $\mathcal{B}(\mathcal{S})$ is the set of bounded measurable functions on $\mathcal{S}$, and p_θ is the trajectory distribution induced by π_θ .

When $\mathcal{S}$ is discrete, μ_θ admits the classical interpretation of an expected visitation frequency:

$$\mu_\theta(s) = \mathbb{E}_{\mathbf{s}_{1:T} \sim p_\theta} \left[\frac{1}{T} \sum_{t=1}^T \mathbb{1}_{\{s_t=s\}} \right].$$

We will not assume $\mathcal{S}$ is discrete in what follows; the discrete case is only meant as intuition. We denote by μ_E the expert occupancy measure, induced by the (unknown) expert policy and the environment dynamics.

State-marginal matching via divergences State-marginal matching IRL seeks a policy whose state occupancy measure μ_θ matches the expert occupancy measure μ_E . Concretely, one minimizes a discrepancy between these probability measures. Several approaches derive objectives specialized to the Kullback–Leibler (KL) divergence, typically via maximum-entropy RL and reward learning (Ziebart et al.; Ni et al., 2020). More generally, one can use the broader class of f -divergences (Csiszár, 1967), which provides a unified way to quantify distribution mismatch.

Definition 3.2 (f -divergence). The f -divergence between two probability measures Q and P , induced by a convex function f satisfying $f(1) = 0$, is

$$D_f(Q \| P) := \mathbb{E}_P \left[f \left(\frac{dQ}{dP} \right) \right]. \quad (1)$$

This family includes KL, Reverse-KL (rKL), and total variation distance. The choice of f matters: different divergences induce different trade-offs (e.g., mode-covering (KL) vs. mode-seeking behavior (rKL)), which can change the qualitative properties of the learned policy (Nowozin et al., 2016; Verine, 2024).

In this work, we consider objectives of the form

$$\min_{\pi_\theta} D_f(\mu_E \| \mu_\theta), \quad (2)$$

subject to the MDP dynamics induced by the sampler.

Prior work and our approach State-marginal matching IRL has often been approached through reward learning under maximum-entropy RL (Ziebart et al.; Ni et al., 2020). In these methods, one first estimates a reward (or energy) from expert states and then relies on MaxEnt RL to induce a policy whose state distribution matches the expert. This indirect route can be undesirable in our setting, as it ties performance to the quality of the learned reward and to the ability of the MaxEnt policy to explore diverse states. By contrast, Ghasemipour et al. (2019) show how to optimize policies *directly* by minimizing f -divergences via convex duality, but their formulation targets *state–action* occupancy measures. To the best of our knowledge, an analogous reward-free, direct minimization of $D_f(\mu_E \| \mu_\theta)$ for *state* marginals has not been developed in the context we consider.

Our approach addresses two gaps: (i) we avoid explicit reward learning by directly optimizing a sampling policy to match expert state distributions, and (ii) we instantiate this objective for diffusion samplers, enabling principled optimization of sampler hyperparameters. In Section 4, we make the diffusion-to-MDP mapping explicit and derive the resulting optimization procedure.

4. Improving Diffusion with IRL

In this section, we present our theoretical contributions and introduce a general framework for controlling generative sequential processes. We first formalize diffusion sampling as a finite-horizon MDP in Section 4.1. We then describe how to learn sampling policies by minimizing f -divergences between state occupancy measures in Section 4.2.

4.1. The MDP Model for Diffusion Sampling

To apply our inverse reinforcement learning algorithm, we frame each previously defined component as an element of

diffusion sampling. The state space $\mathcal{S}$ is continuous, and each state is defined as

$$s_t = (x_t, \sigma_t),$$

where σ_t denotes the noise level at time step t and $x_t \in \mathbb{R}^d$ is the current sample. The action space $\mathcal{A}$ is discrete and encodes control decisions applied to the sampling dynamics. For instance, actions can correspond to selecting different discretized guidance scales, noise injection levels, or whether to renoise and how much at each timestep. The initial distribution ρ corresponds to the marginal distribution at the highest noise level, the Gaussian distribution $\mathcal{N}(0, \Sigma_N^2 I_d)$. Given a state-action pair (s_t, a_t) , the transition kernel $\mathcal{P}$ is induced by the discretized diffusion dynamics together with the selected control action. The policy does not explicitly depend on time but the available actions may depend on the current timestep t . We will treat $\sigma_t = \Sigma_0$ as an absorbing condition, so that once the process reaches this noise level, it remains there for all subsequent timesteps. In practice, this can be implemented by defining H such that $s_{t+1} = s_t$ whenever $\sigma_t = \Sigma_0$. In this setup, the average time required to reach the terminal noise level may be incorporated in the performance metric of a given policy.

As the state space is continuous, we rely on the definition 3.1. Exploiting the structure of the state space, we further decompose the occupancy measure as

$$\mu_\theta(s) = w_\theta(\sigma) \times p_\theta(x|\sigma),$$

where $w_\theta(\sigma)$ denotes the weight assigned by the policy π_θ to noise level Σ , and $p_\theta(x|\sigma)$ is the marginal distribution of x at noise level Σ under the policy. For the samplers such as noise injection or guidance, w_θ is uniform over noise levels, since the process always progresses from Σ_N to Σ_0 in T steps. For samplers with renoising, w_θ reflects the distribution over noise levels induced by the policy, which may spend more time at certain noise levels depending on the restart strategy.

Expert occupancy measure In our framework, the expert is specified through samples from the target distribution at selected noise levels. We do not assume access to expert trajectories, actions, or transition dynamics. The expert occupancy measure μ_E is

$$\mu_E(s) = w_E(\sigma) \times p_E(x|\sigma).$$

As the expert does not provide temporal information, the weights $w_E(\sigma)$ are a design choice that favors trajectories reaching quickly states $s_t = (x_t, \sigma_t)$ such that $\sigma_t = \Sigma_0$. There are various ways to define w_E to reflect this preference. A simple choice is to set it uniformly over $\{\Sigma_N, \dots, \Sigma_1\}$ and assign all remaining weight to Σ_0 . Our primary objective is to match the expert distribution at this terminal noise level. The absorbing condition $\{\sigma_t = \Sigma_0\}$

allows all states at noise level Σ_0 to contribute to the occupancy measure, yielding a denser learning signal that reflects how closely the generated samples match the expert distribution at the lowest noise level. w_E can also be interpreted as a proxy for the time spent at each noise level. This motivates us to interpret $w_E(\Sigma_0)$ as a measure of the mean time spent at the terminal noise level, which we want the policy $w_\theta(\Sigma_0)$ to match.

On the leverage of w_E Given the definitions above, minimizing an f -divergence between the expert and policy-induced occupancy measures implies a corresponding lower bound on the divergence between the marginal noise-level weights:

$$\mathcal{D}_f(\mu_E \parallel \mu_\theta) \geq \mathcal{D}_f(w_E \parallel w_\theta). \quad (3)$$

This follows from the data processing inequality for f -divergences with convex generators. Consequently, the choice of w_E directly influences the optimization problem. Moreover, for the Kullback–Leibler and reverse Kullback–Leibler divergences, the objective admits an explicit decomposition into a term depending only on w_E and w_θ , and a term involving the conditional marginals at each noise level:

$$\begin{aligned} D_{\text{KL}}(\mu_E \parallel \mu_\theta) &= D_{\text{KL}}(w_E \parallel w_\theta) \\ &\quad + \sum_{\sigma} w_E(\sigma) D_{\text{KL}}(p_E(\cdot | \sigma) \parallel p_\theta(\cdot | \sigma)), \\ \mathcal{D}_{\text{rKL}}(\mu_E \parallel \mu_\theta) &= \mathcal{D}_{\text{rKL}}(w_E \parallel w_\theta) \\ &\quad + \sum_{\sigma} w_\theta(\sigma) \mathcal{D}_{\text{rKL}}(p_E(\cdot | \sigma) \parallel p_\theta(\cdot | \sigma)). \end{aligned} \quad (4)$$

This decomposition highlights how the optimization jointly controls the distribution over noise levels and the quality of samples generated at each noise level.

4.2. Learning Policies by Minimizing f -Divergences

We now describe how to optimize sampling policies within the MDP framework introduced above. We directly optimize the policy to match the expert occupancy measure by optimizing the following objective:

$$\min_{\theta} \mathcal{L}_f(\theta) = \min_{\theta} \mathcal{D}_f(\mu_E \parallel \mu_\theta). \quad (6)$$

Policy gradient optimization We show in Appendix A.1 that the gradient of the f -divergence objective with respect to the policy parameters, denoted $\nabla_{\theta} \mathcal{L}_f(\theta)$, can be expressed in the form of a policy gradient.

Theorem 4.1. *Let π_θ be a policy parameterized by θ and let $\mathcal{L}_f(\theta) = \mathcal{D}_f(\mu_E \parallel \mu_\theta)$ be the f -divergence between the expert occupancy measure μ_E and the occupancy measure μ_θ induced by the policy π_θ . Then, the gradient of $\mathcal{L}_f(\theta)$ with respect to θ is given by*

$$\nabla_{\theta} \mathcal{L}_f(\theta) = \mathbb{E}_{s_{1:T} \sim p_\theta} \left[\frac{1}{T} \sum_{t=1}^T \nabla_{\theta} \log \pi_\theta(a_t | s_t) A_t \right], \quad (7)$$

where $A_t = \sum_{t' \geq t} h_f\left(\frac{\mu_E(s_{t'})}{\mu_\theta(s_{t'})}\right)$ is the learning signal, $h_f(x) = f(x) - xf'(x)$ for all x and f is the generator function of the f -divergence (see Appendix A for details).

This result enables the use of standard policy gradient methods to optimize the sampling policy using Monte Carlo estimates obtained from rollouts of the current policy. While this result seems simple and similar to other policy gradient theorems, the main novelty lies in the direct optimization of the policy instead of a reward function.

5. Practical Optimization

In this section, we describe how to estimate the density ratios $\frac{\mu_E}{\mu_\theta}$ appearing in the policy gradient expression in equation 4.1, and how to optimize the policy in practice.

Discriminator-based density ratio estimation Estimating density ratios in continuous spaces is a classical problem, with approaches ranging from kernel density estimation to direct ratio fitting and probabilistic modeling (Sugiyama et al., 2010). In this work, we adopt a classifier-based approach, which has been shown to scale effectively to high-dimensional settings (Ho & Ermon, 2016; Finn et al., 2016; Song & Ermon, 2020; Roth et al., 2017; Azadi et al., 2019) and has successfully been applied to diffusion models (Kim et al., 2023a; Verine et al., 2025).

We train a binary classifier $D_\varphi(x, \sigma)$ to discriminate between states sampled from the expert and from the policy, conditioning on the noise level σ . Since the expert provides only marginal distributions at each noise level, we sample σ uniformly and draw states independently from $p_E(\cdot | \sigma)$ and $p_\theta(\cdot | \sigma)$. The classifier is trained by minimizing the binary cross-entropy loss

$$\mathcal{L}_D(\varphi) = \mathbb{E}_{\sigma \sim \mathcal{U}[\Sigma_0, \dots, \Sigma_N]} \left[-\mathbb{E}_{x \sim p_E(\cdot | \sigma)} [\log D_\varphi(x, \sigma)] - \mathbb{E}_{x \sim p_\theta(\cdot | \sigma)} [\log(1 - D_\varphi(x, \sigma))] \right].$$

Assuming an optimal discriminator, the density ratio is

$$\frac{p_E(x | \sigma)}{p_\theta(x | \sigma)} = \frac{D_\varphi(x, \sigma)}{1 - D_\varphi(x, \sigma)} = \exp(\tilde{D}_\varphi(x, \sigma)), \quad (8)$$

where $\tilde{D}_\varphi(x, \sigma)$ denotes the logit output of the classifier.

Computing the occupancy measure ratio Because expert trajectories are unavailable, the expert is specified only through state marginals at each noise level. As a result, the discriminator estimates a density ratio under a uniform distribution over noise levels rather than under the policy-induced state occupancies. In addition, while the policy accumulates occupancy mass at the terminal noise level, the expert does not provide temporal occupancy information.

We compute the full occupancy measure ratio by using importance sampling corrections to account for the mismatch

between sampling and target proportions.

$$\frac{\mu_E(x, \sigma)}{\mu_\theta(x, \sigma)} = \frac{w_E(\sigma)}{w_\theta(\sigma)} \times \frac{p_E(x | \sigma)}{p_\theta(x | \sigma)}. \quad (9)$$

As previously discussed, the term w_E is a design choice while w_θ is induced by the policy and is estimated empirically from the simulated trajectories.

Sample efficiency and practical considerations Generating trajectories is computationally expensive, as each rollout requires simulating a full diffusion sampling trajectory. To improve sample efficiency, we reuse trajectories over multiple policy updates and add importance sampling corrections to the policy gradient. While this modification is theoretically sound, we found it to be unstable since it leads to high-variance gradient estimates. We attempted to mitigate this issue by subtracting a baseline, a standard technique for reducing variance in policy gradient methods. However, in our setting, there is no clear choice of baseline. We therefore use the empirical average of the learning signals $\bar{A}_t = \frac{1}{n_{\text{traj}}} \sum_{i=1}^{n_{\text{traj}}} A_t^{(i)}$ as a baseline.

Algorithm 1 Policy optimization algorithm

Require: Update Sampler Operator H , pretrained denoiser F , expert state marginals $\{\mu_E(\cdot | \sigma)\}_\sigma$, divergence f

- 1: Initialize sampling policy π_θ
- 2: Initialize state density-ratio estimator D_φ
- 3: **repeat**
- 4: Generate trajectories using π_θ choosing parameters of H and frozen denoiser F
- 5: Sample expert and policy states uniformly across noise levels
- 6: Train D_φ to discriminate expert from policy states
- 7: Compute policy learning signals A_t using D_φ and occupancy weights
- 8: Update π_θ using a $\nabla_\theta \mathcal{L}_f^{\text{PPO}}(\theta)$
- 9: **until** convergence

Ensure: Optimized sampling policy π_θ

To further reduce variance, we adopt a Proximal Policy Optimization (PPO)-style clipped objective (Schulman et al., 2017), which constrains the policy updates at each iteration as well as reward normalization. Under the same assumptions as Theorem 4.1, the PPO-style clipped gradient for minimizing an f -divergence is given by

$$\nabla_\theta \mathcal{L}_f^{\text{PPO}}(\theta) = \mathbb{E}_{p_{\theta_0}} \left[\sum_{t=1}^T \nabla_\theta \max(r_t(\theta) \hat{A}_t, \tilde{r}_t(\theta) \hat{A}_t) \right]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ is the probability ratio between the current and sampling policies, $\tilde{r}_t(\theta) = \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$ is the clipped probability ratio, and $\hat{A}_t = \frac{1}{T}(A_t - \bar{A}_t)$ denotes the recentered learning signal at time t .

Finally, following common practice, we approximate $h_f(\mu_E(s_t)/\mu_\theta(s_t))$ by $h_f(\mu_E(s_t)/\mu_{\theta_0}(s_t))$ to avoid re-computing the density ratio at every policy update. Algorithm 1 summarizes the high-level structure of the optimization procedure. We provide a detailed Algorithm 2 in Appendix B.1.

6. Experiments

We evaluate the proposed framework across several diffusion sampling strategies to address four questions: (i) how broadly the method applies to different modifications of the sampling dynamics, (ii) whether conditioning the policy on the current sample x_t is beneficial, (iii) how sensitive the approach is to initialization, and (iv) how the choice of divergence controls the precision–recall trade-off.

Experiments are conducted on CIFAR-10, FFHQ, and ImageNet using pretrained EDM models (Karras et al., 2022). The expert conditional distribution $p_E(x|\sigma)$ is defined as the empirical dataset distribution at noise level $\Sigma_0 = 0$ with additional Gaussian noise at level σ . All policy generations are evaluated using FID (Heusel et al., 2017) with 50k samples, Precision and Recall (Kynkäänniemi et al., 2019) using the improved topological method of Kim et al. (2023b), and the average Number of Function Evaluations (NFE) required to reach the terminal noise level (when relevant).

What strategies can be applied? A key strength of the framework is its flexibility: it can be applied to a wide range of sampling-time interventions without modifying the denoiser. We consider adaptive classifier-free guidance, Renoise strategy that revisits higher-noise states and learning the stochasticity injection profile γ_{EDM} . Despite their differences, all strategies are learned using the same state-occupancy matching objective.

Computational overhead Our method introduces negligible overhead at sampling time. The learned policies are lightweight neural networks with small action spaces, and their evaluation cost is insignificant compared to denoiser evaluations. The main computational cost arises during training, where multiple sampling trajectories are required to estimate gradients and train the discriminator. This cost remains far below that of retraining the denoiser itself.

6.1. Learning Adaptive Classifier-Free Guidance

Classifier-free guidance trades diversity for fidelity via a guidance scale ω , but fixed guidance schedules have been shown to collapse to the mean of the conditional distribution (Chidambaram et al., 2024). In this experiment, we explore learning an adaptive, state-dependent guidance policy within our framework. Experiments are conducted on CIFAR-10 32×32 and FFHQ 64×64 using pretrained EDM models.

Table 1. Performance comparison for learning the guidance force $\omega(x, \sigma)$. Adaptive guidance improves FID as well as Precision and Recall compared to fixed guidance scales.

Method	Policy	FID ↓	Precision ↑	Recall ↑
CIFAR-10 (32×32)				
KL	$\pi(x, \sigma)$	2.05	80.7	71.4
	$\pi(\sigma)$	2.26	80.2	70.9
rKL	$\pi(x, \sigma)$	2.53	82.4	69.8
	$\pi(\sigma)$	2.23	80.2	70.9
EDM + CFG ($\omega = 0.2$)	X	2.07	80.6	70.9
FFHQ (64×64)				
KL	$\pi(x, \sigma)$	3.05	90.4	88.6
rKL	$\pi(x, \sigma)$	3.04	90.7	88.3
EDM + CFG ($\omega = 0.1$)	X	3.11	90.7	87.8

We compare the learned adaptive guidance against standard fixed guidance scales. We also use this experiment as a way to compare policies dependent on the image state $\pi_\theta(x, \sigma)$ against simpler policies dependent only on the noise level $\pi_\theta(\sigma)$ while using the same initialization for both policies. Quantitative results are reported in Table 1. The details about x -dependent policies are provided in Appendix B.2.

Although KL and rKL achieve comparable FID scores, they do so through qualitatively different guidance profiles. KL places more emphasis on covering the support of the data distribution, improving recall and encouraging sample diversity. In contrast, rKL concentrates probability mass on high-density regions, favoring precision and higher-fidelity samples. Figure 2b illustrates how these two different options impact the learned guidance strength profiles. The behavior exhibited by the learned policies consists in using high guidance for a small proportion of the images at each timestep, while leaving the majority of images unguided. The policy increases the proportion of highly guided sam-

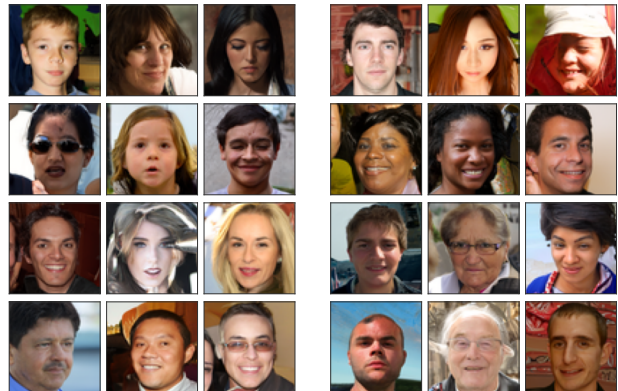


Figure 1. FFHQ samples generated with fixed guidance $\omega = 0.1$ (left) and with adaptive guidance optimized with rKL (right).

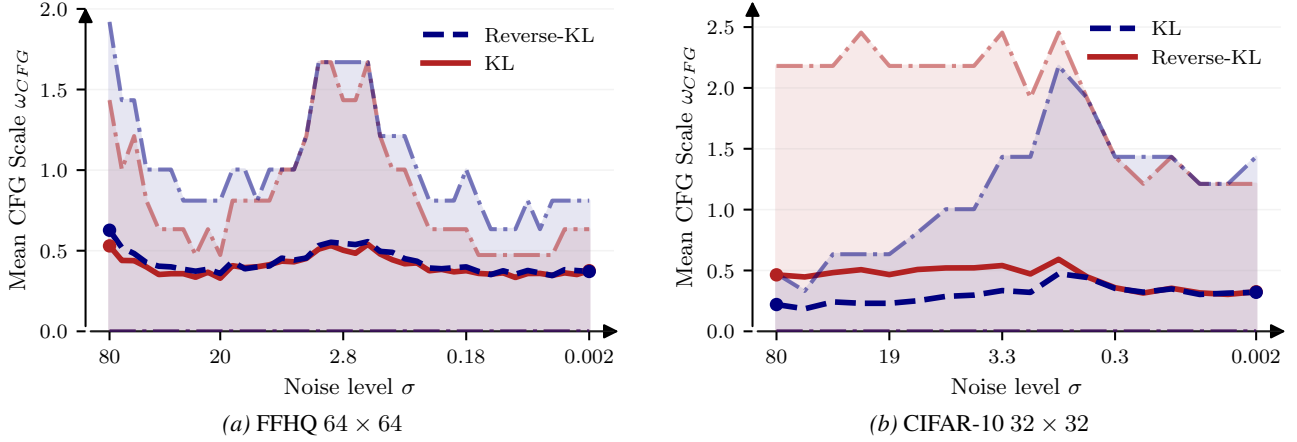


Figure 2. Learned classifier-free guidance profiles $\omega(x, \sigma)$ for different f -divergences. Higher guidance improves precision but reduces diversity. We report the mean as well as the 0.25 and 0.75 quantiles across samples.

ples in the medium-to-low noise levels, corresponding to the speciation phase of sampling proposed by Biroli et al. (2024). Around medium noise levels, the guidance strength exhibits higher mean and variance, indicating a greater need for guidance during this phase, which coincides with the emergence of topological structures in the images. Moreover, the dependence in x leads to policies with improved performance. This experiment highlights two important points. First, conditioning the policy on the current state (x, σ) is crucial, as a single scalar guidance value cannot capture the varying trade-offs required at different noise levels. Second, the choice of divergence provides a direct and interpretable mechanism for controlling the precision-recall trade-off during sampling.

6.2. Learning Adaptive Renoising

We next use the empirical dataset distribution as the expert to improve the EDM sampler through renoising and stochasticity injection. Experiments are conducted on CIFAR-10 32×32 , FFHQ 64×64 , and ImageNet 64×64 .

Learning to renoise Renoising consistently improves performance on CIFAR-10 and ImageNet, highlighting the fact that going beyond fixed noise schedules can be beneficial. On CIFAR-10, performance depends strongly on the choice of divergence. KL-based objectives improve sample quality while reducing NFE, whereas Reverse-KL tends to allocate additional computation to high-noise regions that have limited influence on the final samples, which results in increased NFE without gains in FID. This observation shows that the choice of divergence influences not only the objective being optimized, but also the points along the sampling trajectory where the learned policy is most active.

Learning γ_{EDM} We now study learning the stochasticity injection profile $\gamma_{\text{EDM}}(x, \sigma)$. Learning γ_{EDM} consistently matches or improves upon the baseline FID across datasets.

Table 2. Performance comparison for learning renoise strategies. Best results are highlighted in bold.

Method	Policy	FID	Precision	Recall	NFE
CIFAR-10 (32×32)					
KL	$\pi(x, \sigma)$	3.18	77.3	72.5	17.7
	$\pi(\sigma)$	3.45	77.4	71.7	17.0
rKL	$\pi(x, \sigma)$	3.21	78.1	71.6	30.5
	$\pi(\sigma)$	3.46	76.5	72.2	26.5
EDM	$\times$	3.42	76.7	72.5	17
FFHQ (64×64)					
KL	$\pi(x, \sigma)$	3.04	90.9	88.1	87.4
rKL	$\pi(x, \sigma)$	3.07	90.9	87.9	68.2
EDM	$\times$	2.60	90.5	89.0	59
ImageNet (64×64)					
KL	$\pi(x, \sigma)$	2.96	85.4	92.3	51.8
rKL	$\pi(x, \sigma)$	2.92	85.4	92.1	52.1
EDM	$\times$	3.01	85.8	91.8	350

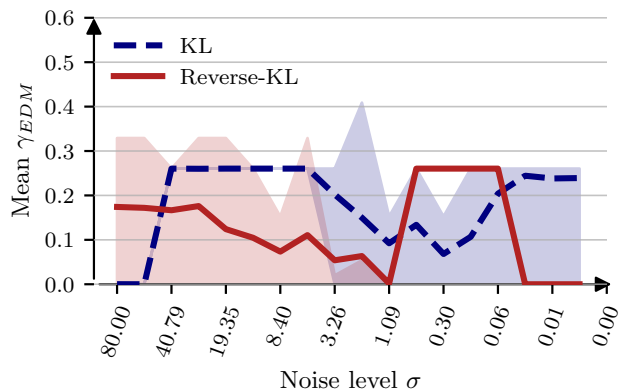


Figure 3. Learned stochasticity injection profiles $\gamma_{\text{EDM}}(x, \sigma)$ on CIFAR-10 32×32 for different f -divergences.

Table 3. Performance comparison for learning the stochasticity injection profile $\gamma_{\text{EDM}}(x, \sigma)$. Best results are highlighted in bold.

Method	Policy	FID	Precision	Recall
CIFAR-10 (32×32)				
KL	$\pi(x, \sigma)$	1.99	78.5	73.2
	$\pi(\sigma)$	2.03	78.7	72.7
rKL	$\pi(x, \sigma)$	2.03	79.7	71.9
	$\pi(\sigma)$	2.03	78.6	72.4
EDM	X	1.98	78.7	72.9
FFHQ (64×64)				
KL	$\pi(x, \sigma)$	2.53	90.4	88.2
rKL	$\pi(x, \sigma)$	2.49	90.9	87.9
EDM	X	2.56	90.1	89.4
ImageNet (64×64)				
KL	$\pi(x, \sigma)$	2.14	85.7	92.0
rKL	$\pi(x, \sigma)$	2.24	86.3	91.1
EDM	X	2.12	85.9	91.7

More importantly, the behavior of the divergences is consistent with previous experiments. In this experiment as well, conditioning on x reduces policy stochasticity; however, the induced behavior does not exhibit a simple or easily predictable structure, highlighting the importance of x -dependence for this task. On CIFAR-10, the Reverse-KL and KL have totally different behaviors with the Reverse-KL adding less noise with low variance at low-noise regions to increase precision. The KL is the opposite adding more noise with higher variance to increase diversity. Figure 3 illustrates the learned profiles.

6.3. Impact of Temperature on the Policy

Temperature is a standard mechanism for controlling the trade-off between sample diversity and fidelity, notably in the context of large language models. In our framework, the sampling policy is stochastic and draws actions from a categorical distribution, which allows us to directly control the entropy of the policy by adjusting its temperature at inference time. Letting β denote the temperature parameter, the action a_t at time t is sampled according to $a_t \sim \pi_\theta(a_t | s_t)^{1/\beta} / Z$, where Z is the normalizing constant. Lower temperatures concentrate probability mass on high-likelihood actions, yielding more deterministic behavior, while higher temperatures flatten the distribution and encourage exploration. We study the impact of temperature on learned policies for noise injection on CIFAR-10 32×32 . As shown in Figure 4, increasing the temperature leads to a higher number of function evaluations (NFE), as the policy more frequently selects actions that inject noise and revisit higher-noise states. This increased exploration results in longer trajectories, but has a limited effect on sample quality. In particular, Precision and Recall remain

relatively stable across a wide range of temperatures, indicating that the learned policy is robust to moderate changes in entropy at sampling time. A similar behavior is observed for policies learned to control the stochasticity injection parameter γ_{EDM} (see Appendix B.3). In contrast, it is observed that trying to enforce a more deterministic behavior of the learned policy (i.e., using smaller β) is detrimental in terms of image quality (as measured by FID). This observation suggests that deterministic schedules, that are commonly used in practice, may be inherently sub-optimal.

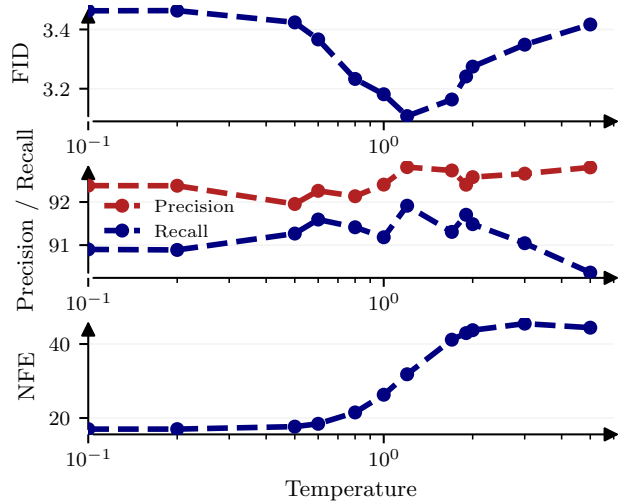


Figure 4. Impact of temperature on Precision, Recall, and NFE for learned renoise policies on CIFAR-10 32×32 . Higher temperature increases diversity and NFE.

7. Conclusion

We introduced an inference-time framework for adapting diffusion sampling by optimizing sampling policies via inverse reinforcement learning, without retraining the denoiser or learning explicit rewards. By matching expert state occupancy measures through f -divergence minimization, the method provides an interpretable way to control sampling behavior. Experiments on CIFAR-10, FFHQ, and ImageNet indicate improved cost-quality trade-offs across multiple sampling strategies.

While our framework provides a flexible and effective approach for improving diffusion sampling, it currently has some limitations. First, our experiments show sensitivity to initialization, as poorly initialized policies can converge to suboptimal local minima, highlighting the need for better heuristics or pretraining. Second, performance depends strongly on the design of the discrete action space, with poorly chosen actions degrading sampling quality. Exploring continuous action spaces may alleviate this issue for some strategies and is left for future work.

Acknowledgements

This work was granted access to the HPC resources of IDRIS under the allocations 2025-AD011016159R1, 2025-A0181016159 and 2025-AD011017138 made by GENCI and was supported by the PEPR Santé Numérique - SSF-ML-DH project.

References

- Alex Krizhevsky. Learning multiple layers of features from tiny images. 2009.
- Azadi, S., Olsson, C., Darrell, T., Goodfellow, I., and Odena, A. Discriminator Rejection Sampling, February 2019. URL <http://arxiv.org/abs/1810.06758>. arXiv:1810.06758 [cs, stat].
- Azangulov, I., Potapchik, P., Li, Q., Aamari, E., Deligiannidis, G., and Rousseau, J. Adaptive diffusion guidance via stochastic optimal control, 2025. URL <https://arxiv.org/abs/2505.19367>.
- Azar, M. G., Rowland, M., Piot, B., Guo, D., Calandriello, D., Valko, M., and Munos, R. A general theoretical paradigm to understand learning from human preferences, 2023. URL <https://arxiv.org/abs/2310.12036>.
- Barceló, R., Alcázar, C., and Tobar, F. Avoiding mode collapse in diffusion models fine-tuned with reinforcement learning, 2024. URL <https://arxiv.org/abs/2410.08315>.
- Biroli, G., Bonnaire, T., Bortoli, V. d., and Mézard, M. Dynamical Regimes of Diffusion Models, February 2024. URL <http://arxiv.org/abs/2402.18491>. arXiv:2402.18491.
- Black, K., Janner, M., Du, Y., Kostrikov, I., and Levine, S. Training diffusion models with reinforcement learning. *arXiv preprint arXiv:2305.13301*, 2023.
- Chidambaram, M., Gatmiry, K., Chen, S., Lee, H., and Lu, J. What does guidance do? a fine-grained analysis in a simple setting. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 84968–85005. Curran Associates, Inc., 2024. doi: 10.52202/079017-2698.
- Csiszár, I. Information-type measures of difference of probability distributions and indirect observations. *Studia Scientiarum Mathematicarum Hungarica*, 2:299–318, 1967.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, June 2009. doi: 10.1109/CVPR.2009.5206848. URL <https://ieeexplore.ieee.org/document/5206848>. ISSN: 1063-6919.
- Dhariwal, P. and Nichol, A. Diffusion Models Beat GANs on Image Synthesis, June 2021. URL <http://arxiv.org/abs/2105.05233>. arXiv:2105.05233 [cs, stat].
- Domingo-Enrich, C., Drozdal, M., Karrer, B., and Chen, R. T. Q. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control, 2025. URL <https://arxiv.org/abs/2409.08861>.
- Fan, Y., Watkins, O., Du, Y., Liu, H., Ryu, M., Boutilier, C., Abbeel, P., Ghavamzadeh, M., Lee, K., and Lee, K. Reinforcement learning for fine-tuning text-to-image diffusion models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- Finn, C., Christiano, P., Abbeel, P., and Levine, S. A Connection between Generative Adversarial Networks, Inverse Reinforcement Learning, and Energy-Based Models, November 2016. URL <http://arxiv.org/abs/1611.03852>. arXiv:1611.03852 [cs].
- Ghasemipour, S. K. S., Zemel, R., and Gu, S. A Divergence Minimization Perspective on Imitation Learning Methods, November 2019. URL <http://arxiv.org/abs/1911.02256>. arXiv:1911.02256 [cs].
- Guo, J., Chai, W., Deng, J., Huang, H.-W., Ye, T., Xu, Y., Zhang, J., Hwang, J.-N., and Wang, G. Versat2i: Improving text-to-image models with versatile reward. *arXiv preprint arXiv:2403.18493*, 2024.
- Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., and Hochreiter, S. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/8ald694707eb0fefe65871369074926d-Abstract.html>.
- Ho, J. and Ermon, S. Generative Adversarial Imitation Learning, June 2016. URL <http://arxiv.org/abs/1606.03476>. arXiv:1606.03476 [cs].
- Ho, J. and Salimans, T. Classifier-free diffusion guidance, 2022. URL <https://arxiv.org/abs/2207.12598>.

- Ho, J., Jain, A., and Abbeel, P. Denoising Diffusion Probabilistic Models. arXiv, December 2020. URL <http://arxiv.org/abs/2006.11239>. arXiv:2006.11239, NeurIPS 2020.
- Jolicœur-Martineau, A., Li, K., Piché-Taillefer, R., Kachman, T., and Mitliagkas, I. Gotta Go Fast When Generating Data with Score-Based Models, May 2021. URL <http://arxiv.org/abs/2105.14080>. arXiv:2105.14080 [cs, math, stat].
- Karras, T., Laine, S., and Aila, T. A Style-Based Generator Architecture for Generative Adversarial Networks, March 2019. URL <http://arxiv.org/abs/1812.04948>. arXiv:1812.04948 [cs, stat].
- Karras, T., Aittala, M., Aila, T., and Laine, S. Elucidating the Design Space of Diffusion-Based Generative Models, October 2022. URL <http://arxiv.org/abs/2206.00364>. arXiv:2206.00364, NeurIPS 2022.
- Kim, D., Kim, Y., Kwon, S. J., Kang, W., and Moon, I.-C. Refining Generative Process with Discriminator Guidance in Score-based Diffusion Models. April 2023a. URL <http://arxiv.org/abs/2211.17091>. arXiv:2211.17091, ICML 2023.
- Kim, P. J., Jang, Y., Kim, J., and Yoo, J. TopP&R: Robust Support Estimation Approach for Evaluating Fidelity and Diversity in Generative Models, June 2023b. URL <http://arxiv.org/abs/2306.08013>. arXiv:2306.08013 [cs].
- Kynkäänniemi, T., Karras, T., Laine, S., Lehtinen, J., and Aila, T. Improved Precision and Recall Metric for Assessing Generative Models. In *33rd Conference on Neural Information Processing Systems (NeurIPS 2019)*, Vancouver, Canada., October 2019. arXiv: 1904.06991.
- Lee, H., Lu, J., and Tan, Y. Convergence for score-based generative modeling with polynomial complexity, May 2023. URL <http://arxiv.org/abs/2206.06227>. arXiv:2206.06227 [cs, math, stat].
- Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., and Zhu, J. DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps, October 2022. URL <http://arxiv.org/abs/2206.00927>. arXiv:2206.00927 [cs].
- Ni, T., Sikchi, H., Wang, Y., Gupta, T., Lee, L., and Eysenbach, B. f-IRL: Inverse Reinforcement Learning via State Marginal Matching, December 2020. URL <http://arxiv.org/abs/2011.04709>. arXiv:2011.04709, CoRL 2020.
- Nowozin, S., Cseke, B., and Tomioka, R. f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization, June 2016. URL <http://arxiv.org/abs/1606.00709>. arXiv:1606.00709 [cs, stat].
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. Direct Preference Optimization: Your Language Model is Secretly a Reward Model, December 2023. URL <http://arxiv.org/abs/2305.18290>. arXiv:2305.18290 [cs].
- Roth, K., Lucchi, A., Nowozin, S., and Hofmann, T. Stabilizing Training of Generative Adversarial Networks through Regularization. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/7bccfde7714alebdf06c5f4cea752c1-Abstract.html>.
- Saharia, C., Chan, W., Saxena, S., Li, L., Whang, J., Denton, E., Ghasemipour, S. K. S., Ayan, B. K., Mahdavi, S. S., Lopes, R. G., Salimans, T., Ho, J., Fleet, D. J., and Norouzi, M. Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding, May 2022. URL <http://arxiv.org/abs/2205.11487>. arXiv:2205.11487 [cs].
- Santi, R. D., Vlastelica, M., Hsieh, Y.-P., Shen, Z., He, N., and Krause, A. Provable maximum entropy manifold exploration via diffusion models, 2025. URL <https://arxiv.org/abs/2506.15385>.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms, August 2017. URL <http://arxiv.org/abs/1707.06347>. arXiv:1707.06347 [cs].
- Song, J. and Ermon, S. Bridging the Gap Between f-GANs and Wasserstein GANs. In *Proceedings of the 37th International Conference on Machine Learning*, pp. 9078–9087. PMLR, November 2020. URL <https://proceedings.mlr.press/v119/song20a.html>. ISSN: 2640-3498.
- Song, J., Meng, C., and Ermon, S. Denoising Diffusion Implicit Models. October 2022. URL <http://arxiv.org/abs/2010.02502>. arXiv:2010.02502, ICLR 2022.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. SCORE-BASED GENERATIVE MODELING THROUGH STOCHASTIC DIFFERENTIAL EQUATIONS. 2021.
- Sugiyama, M., Suzuki, T., and Kanamori, T. Density Ratio Estimation: A Comprehensive Review. 2010.

- Sun, H. and van der Schaar, M. Inverse reinforcement learning meets large language model post-training: Basics, advances, and opportunities, 2025. URL <https://arxiv.org/abs/2507.13158>.
- Uehara, M., Zhao, Y., Black, K., Hajiramezanali, E., Scalia, G., Diamant, N. L., Tseng, A. M., Biancalani, T., and Levine, S. Fine-tuning of continuous-time diffusion models as entropy-regularized control. *arXiv preprint arXiv:2402.15194*, 2024.
- Verine, A. *Quality and Diversity in Generative Models through the lens of f -divergences*. PhD thesis, January 2024. URL <https://theses.fr/279916922>.
- Verine, A., Inane, A. M., Bronnec, F. L., Negrevergne, B., and Chevalayre, Y. Improving Discriminator Guidance in Diffusion Models, June 2025. URL <http://arxiv.org/abs/2503.16117>. arXiv:2503.16117 [cs].
- Wang, C., Jiang, Y., Yang, C., Liu, H., and Chen, Y. Beyond Reverse KL: Generalizing Direct Preference Optimization with Diverse Divergence Constraints, September 2023. URL <https://arxiv.org/pdf/2405.15624>. arXiv:2309.16240.
- Xu, Y., Deng, M., Cheng, X., Tian, Y., Liu, Z., and Jaakkola, T. Restart Sampling for Improving Generative Processes, November 2023. URL <http://arxiv.org/abs/2306.14878>. arXiv:2306.14878, NeurIPS 2023.
- Yoon, S., Hwang, H., Kwon, D., Noh, Y.-K., and Park, F. C. Maximum entropy inverse reinforcement learning of diffusion models with energy-based models, 2024. URL <https://arxiv.org/abs/2407.00626>.
- Ziebart, B. D., Maas, A., Bagnell, J. A., and Dey, A. K. Maximum Entropy Inverse Reinforcement Learning.

A. Proofs

A.1. Gradient formula proof

Proof. Let's consider our objective function:

$$\mathcal{L}_f(\theta) = \mathcal{D}_f(\mu_E \parallel \mu_\theta)$$

We can compute the gradient of $\mathcal{L}_f(\theta)$ with respect to θ using the chain rule and the properties of the f -divergence:

$$\begin{aligned} \nabla_\theta \mathcal{L}_f(\theta) &= \nabla_\theta \mathcal{D}_f(\mu_E \parallel \mu_\theta) \\ &= \nabla_\theta \int_{\mathcal{S}} f\left(\frac{\mu_E(s)}{\mu_\theta(s)}\right) \mu_\theta(s) ds \\ &= \int_{\mathcal{S}} \nabla_\theta \mu_\theta(s) f\left(\frac{\mu_E(s)}{\mu_\theta(s)}\right) - \mu_\theta(s) \frac{\nabla_\theta \mu_\theta(s)}{\mu_\theta(s)^2} \mu_E(s) f\left(\frac{\mu_E(s)}{\mu_\theta(s)}\right) ds \\ &= \int_{\mathcal{S}} \nabla_\theta \mu_\theta(s) h_f\left(\frac{\mu_E(s)}{\mu_\theta(s)}\right) ds \\ &= \int_{\mathcal{S}} \nabla_\theta \log \mu_\theta(s) h_f\left(\frac{\mu_E(s)}{\mu_\theta(s)}\right) \mu_\theta(s) ds \end{aligned}$$

with $h_f(x) = f(x) - x \dot{f}(x)$.

Now, using the property of the state occupancy measure, we can write:

$$\begin{aligned} \nabla_\theta \mathcal{L}_f(\theta) &= \mathbb{E}_{\mathbf{s}_{1:T} \sim p_\theta} \left[\frac{1}{T} \sum_{t'=1}^T \nabla_\theta \log \mu_\theta(s_{t'}) h_f\left(\frac{\mu_E(s_{t'})}{\mu_\theta(s_{t'})}\right) \right] \\ &= \mathbb{E}_{\mathbf{s}_{1:T} \sim p_\theta} \left[\frac{1}{T} \sum_{t'=1}^T \left(\sum_{t=1}^{t'} \nabla_\theta \log \pi_\theta(a_t | s_t) \right) h_f\left(\frac{\mu_E(s_{t'})}{\mu_\theta(s_{t'})}\right) \right] \\ &= \mathbb{E}_{\mathbf{s}_{1:T} \sim p_\theta} \left[\frac{1}{T} \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \sum_{t' \geq t} h_f\left(\frac{\mu_E(s_{t'})}{\mu_\theta(s_{t'})}\right) \right] \end{aligned}$$

by reindexing the sums. □

A.2. Adding importance sampling

Proof. Adding importance sampling to the gradient formula derived in Theorem 4.1 gives:

$$\nabla_\theta \mathcal{L}_f(\theta) = \mathbb{E}_{\mathbf{s}_{1:T} \sim p_{\theta_0}} \left[\frac{p_\theta(\mathbf{s}_{1:T})}{p_{\theta_0}(\mathbf{s}_{1:T})} \frac{1}{T} \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \sum_{t' \geq t} h_f\left(\frac{\mu_E(s_{t'})}{\mu_\theta(s_{t'})}\right) \right]$$

where θ_0 is the parameter of the policy used to sample the trajectories. □

B. Experimental Details

We parameterize the sampling policy and the state density estimator using a shared neural architecture, implemented as a dual-head network based on an EDM-style Encoder-UNet. The resulting model enables joint representation learning while keeping the policy and density objectives decoupled.

Backbone and Preconditioning Given a noisy state x and noise level σ , we apply the standard EDM preconditioning and map σ to the corresponding normalized sigma variable τ . The network is conditioned on τ and, when applicable, on class labels for conditional generation. For non-stationary policies, the discrete trajectory step index is additionally provided as an

explicit conditioning variable. We also use a pretrained ADM encoder as a frozen feature extractor. The input image is first processed by the ADM encoder, and the resulting latent representation is used as input to the policy and density heads. All ADM parameters are kept fixed throughout training.

Policy Head The policy head outputs logits over a discrete action space of size A , where each action corresponds to a sampling-time control operation (e.g., continuing denoising or restarting to a previous noise level). The policy head is implemented as convolutional layers with output channels equal to A , sharing the same architectural design and conditioning mechanisms as the density head. For stationary policies, the policy is conditioned only on the current state and noise level. For non-stationary policies, the trajectory step index is included as an additional conditioning signal. During training and inference, actions are sampled from the resulting categorical distribution.

Density Head The density head estimates a scalar quantity associated with the state distribution, used for density-ratio-based training objectives. It is implemented as convolutional layers with a single scalar output. The density head shares the same backbone structure and conditioning as the policy head but has independent parameters in the output layers. The output layers of the density head are initialized using Xavier initialization to stabilize early training.

B.1. Training Details

Initialization The initialization of both the policy and density networks plays an important role in practice. Since the learning problem is non-convex, we do not guarantee convergence to a unique policy, and different initializations may lead to different local optima. We initialize the policy network using a task-specific heuristic corresponding to a reasonable sampling strategy (for instance favoring continuation of denoising at high noise levels and allowing restarts at later stages). This initialization provides a stable starting point and significantly improves training stability compared to random initialization.

The density network is initialized using expert trajectories and rollouts generated by the initialized policy. These expert samples are used to initialize the density-ratio estimator before joint training with the policy, which helps stabilize the early stages of optimization.

Optimization For both the policy and density networks, we apply an exponential moving average (EMA) to the network parameters during training. EMA is used for evaluation and inference, as it consistently yields more stable behavior and improved performance. Finally, when training the density estimator, we apply label smoothing at high noise levels. At large noise scales, the input states contain limited information, and hard labels can lead to unstable gradients. Label smoothing in this regime improves robustness and reduces variance during training.

We provide details of the complete training procedure in table 4.

Table 4. Parameters used for training the sampling policies across different datasets and strategies.

Parameters	Renoise			Gammas			Guidance	
	CIFAR-10	FFHQ 64×64	ImageNet 64×64	CIFAR-10	FFHQ 64×64	ImageNet 64×64	CIFAR-10	FFHQ 64×64
Cond. model	×	×	✓	×	×	✓	✓	✓
LR	1e-4	1e-5	1e-5	1e-4	1e-5	1e-5	1e-4	1e-5
# traj.	1024	1536	2048	1024	1536	2048	1024	1536
# DRE Init.	100	200	200	100	200	200	100	200
Action dim.	4	4	4	10	10	20	20	20
Res.	32×32	64×64	64×64	32×32	64×64	64×64	32×32	64×64
# params.	5.8M	5.8M	6.8M	5.8M	5.8M	6.8M	5.8M	5.8M
# Noise levels	9	20	20	18	40	256	18	40
GPU hrs	30	48	80	60	80	160	100	150
GPU	V100	V100	H100	V100	V100	H100	V100	V100

Algorithm 2 Policy Optimization via State-Marginal f-Divergence Minimization

Require: Pretrained denoiser F , policy π_θ , discriminator D_φ , replay buffer B , number of epochs n_{epoch} , batch size N_{batch} , expert state samples $\mathcal{S}_E$, clipping parameter ϵ , learning rate η , and policy update frequency K .

1: Initialize policy π_θ and discriminator D_φ .

2: **for** $k = 1$ **to** n_{epoch} **do**

3: **if** $k \bmod K = 0$ **then**

4: Generate trajectories

$$\mathbf{s}_{1:T} = \{(s_t, a_t)\}_{t=1}^T$$

 using policy π_θ and denoiser F .

5: Train discriminator D_φ to distinguish expert states $\mathcal{S}_E$ from policy-generated states $\{s_t\}$.

6: Estimate density ratios using

$$\frac{\mu_E(s)}{\mu_\theta(s)} \approx \frac{D_\varphi(s)}{1 - D_\varphi(s)}.$$

7: Compute advantage estimates

$$A_t = \frac{1}{T} \sum_{t'=t}^T h_f \left(\frac{\mu_E(s_{t'})}{\mu_\theta(s_{t'})} \right), \quad h_f(x) = f(x) - x f'(x),$$

 and store (s_t, a_t, A_t) in buffer B .

8: Save current policy parameters: $\theta_0 \leftarrow \theta$.

9: **end if**

10: **for** each minibatch $\{(s_i, a_i, A_i)\}_{i=1}^{N_{\text{batch}}}$ sampled from B **do**

11: Compute PPO-style objective:

$$\hat{\mathcal{L}}_f(\theta) = \frac{1}{N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \max(r_i(\theta) (A_i - \bar{A}), \text{clip}(r_i(\theta), 1 - \epsilon, 1 + \epsilon) (A_i - \bar{A})),$$

 where

$$r_i(\theta) = \frac{\pi_\theta(a_i | s_i)}{\pi_{\theta_0}(a_i | s_i)}, \quad \bar{A} = \frac{1}{N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} A_i.$$

12: Update policy parameters:

$$\theta \leftarrow \theta - \eta \nabla_\theta \hat{\mathcal{L}}_f(\theta).$$

13: **end for**

14: **end for**

Ensure: Trained sampling policy π_θ .

B.2. Is it necessary to have x -dependent policies for CFG?

In this appendix, we analyze whether conditioning the sampling policy on the image state x is necessary when learning adaptive classifier-free guidance (CFG). In our framework, this corresponds to comparing policies of the form $\pi_\theta(\omega | x, \sigma)$ against simpler policies that depend only on the noise level, $\pi_\theta(\omega | \sigma)$.

A noise-level-dependent policy learns a single guidance scale per noise level, which can be interpreted as a global trade-off between fidelity and diversity at each stage of the sampling process. While this is sufficient to recover standard hand-tuned guidance schedules, it cannot adapt guidance strength across samples. In contrast, an x -dependent policy can modulate the guidance scale based on the current state of the sample, allowing different images to receive different amounts of guidance at the same noise level.

Empirically, we observe that x -dependent policies consistently outperform σ -only policies in terms of sample quality. As reported in Table 1, conditioning on (x, σ) improves FID, Precision, and Recall across datasets and divergences. Qualitatively, the learned policies exhibit sparse behavior: at a given noise level, only a subset of samples receives strong

guidance, while the majority remain weakly guided. This behavior cannot be captured by a single scalar guidance value shared across all samples.

This observation aligns with recent analyses of CFG, which suggest that excessive guidance can lead to mode collapse and loss of diversity, while insufficient guidance degrades fidelity. An x -dependent policy enables the sampler to selectively apply strong guidance where it is most beneficial, while preserving diversity elsewhere. In practice, this adaptivity appears crucial for achieving favorable precision–recall trade-offs without manual tuning.

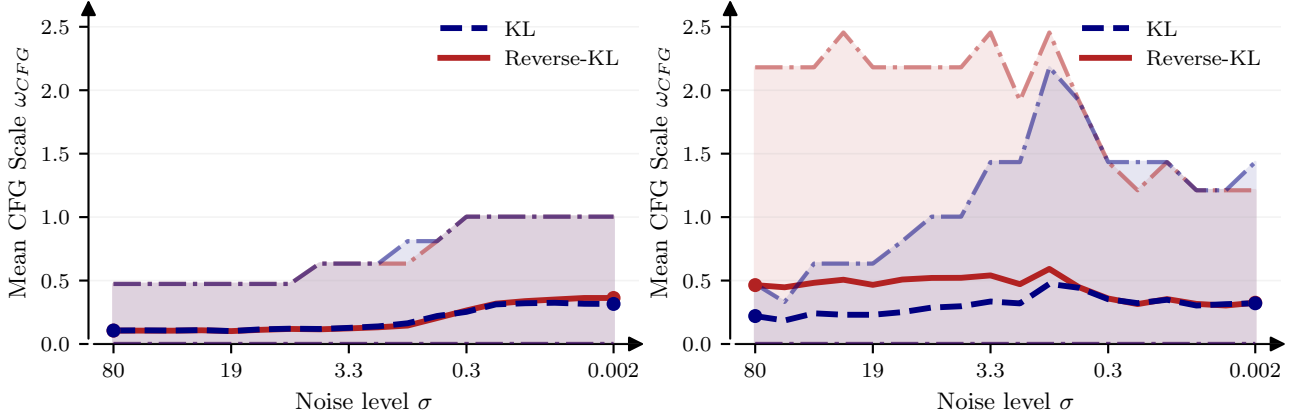


Figure 5. Comparison of x -dependent and noise-level–dependent policies for adaptive CFG on CIFAR-10 using KL and Reverse-KL divergence minimization. The x -dependent policy achieves better FID, Precision, and Recall by selectively applying guidance based on the current sample state. The left profile corresponds to a policy depending only on noise level σ , while the right profile uses both x and σ . It is clear that the policy independent of x produces suboptimal results.

We also trained an x -independent policy for the Gamma strategy on CIFAR-10, but it also underperformed compared to the x -dependent version, suggesting that adaptivity to the current sample state is generally beneficial across different sampling interventions.

Table 5. Comparison of x -dependent and noise-level–dependent policies for adaptive Gamma sampling on CIFAR-10 using KL and Reverse-KL divergence minimization. The x -dependent policy achieves better FID, Precision, and Recall by selectively adjusting the noise schedule based on the current sample state.

Method	FID ↓	Precision ↑	Recall ↑	x -dependent
KL	1.99	78.5	73.2	×
	2.03	78.6	72.7	✓
	2.06	79.0	72.4	✓
rKL	2.03	79.7	71.9	×
	2.03	78.6	72.4	✓
	2.06	79.2	72.5	✓
EDM	1.98	78.7	72.9	×

B.3. Temperature

We experimented with adjusting the temperature of the policy during evaluation to study its effect on the trade-off between sample quality and diversity. Lowering the temperature sharpens the action distribution, leading to more deterministic behavior, while increasing it encourages exploration by flattening the policy over noise-injection actions. As discussed in Section 5.2, temperature therefore provides an inference-time mechanism to control the exploration–exploitation balance of the learned policy.

Figure 6 shows that increasing temperature leads to higher Recall but also higher FID. For the Gamma strategy, this behavior can be attributed to more frequent stochasticity injection at higher noise levels, which increases sample diversity but also

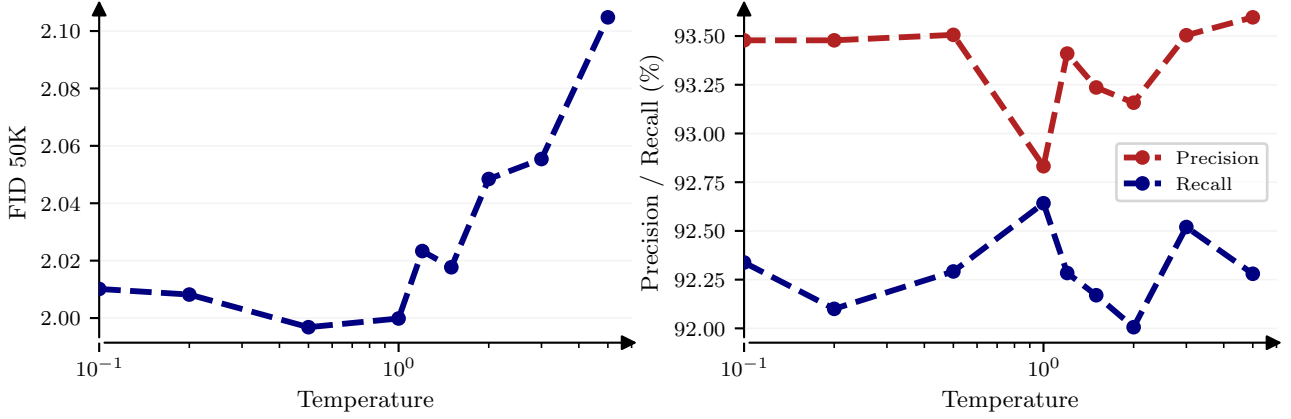


Figure 6. Effect of temperature scaling on the learned Gamma sampling policy for CIFAR-10 using KL divergence minimization.

perturbs trajectories that would otherwise converge to high-fidelity modes. As a result, diversity gains come at the cost of reduced sample sharpness.

Notably, we observe that Precision and Recall are jointly optimized near the temperature used during training ($\beta = 1$). This suggests that the learned policy is already calibrated to balance exploration and exploitation under the training objective, and that deviating significantly from this temperature primarily trades off fidelity for diversity rather than uncovering strictly better operating points.

B.4. Controlling the Distribution of Generated Samples

We also considered settings where the output distribution of a pretrained model is biased and must be corrected at inference time. We focused on renoising strategies. On CIFAR-10, although the dataset is balanced, pretrained EDM models overproduce animal classes relative to vehicle classes. We defined a target class distribution that increases the mass of vehicle classes while preserving normalization. Figure 7 shows that the learned renoise policy shifts the generated class distribution toward the target. While the match is not exact, the bias is substantially reduced, indicating that the policy captures the intended correction direction.

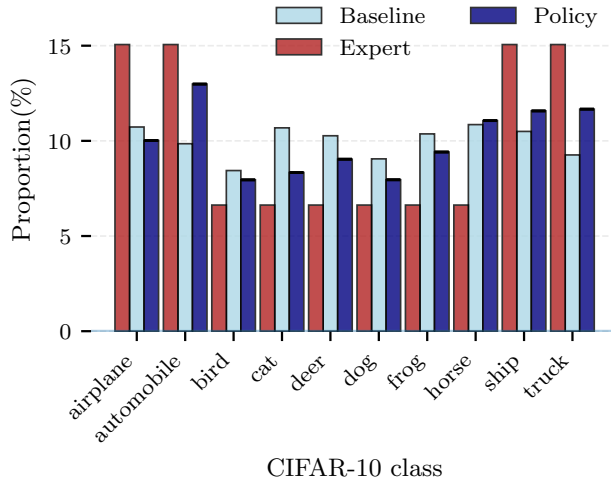


Figure 7. Class distribution of samples generated by a pretrained EDM model on CIFAR-10, before and after applying a learned renoise policy to correct class imbalance. The target distribution increases the mass of vehicle classes (automobile, truck) relative to animal classes. The learned policy shifts the generated distribution closer to the target, reducing bias while maintaining sample quality.

A key modeling choice is whether the control policy depends explicitly on the trajectory step of the induced MDP. We distinguish between non-stationary policies $\pi_\theta(x, \sigma, t)$, which depend on the step index, and stationary policies $\pi_\theta(x, \sigma)$,

which don't. Although the optimal policy for a finite sampling horizon is generally non-stationary, we find that stationary policies are significantly easier to train and often perform better in practice. Across all strategies, stationary policies consistently match or outperform non-stationary ones, yielding more stable optimization and better trade-offs between distributional control and sample quality at comparable NFE.